Universidad Nacional de Mar del Plata
Ingeniería en Informática

Trabajo Práctico Final

Estrategia de Testing

Aplicación: Clínica

Materia
Calidad de Software

Integrantes
Lucas Cardozo
Agustín Zalazar

Repositorio
github.com/LucasIvanCardozo/Testing-G11-CalidadDeSoftware

Octubre de 2026

Contenido

1. Introducción . . . 3
   1.1 Entorno . . . 3
   1.2 Cómo se ejecuta la suite . . . 3
2. Resumen de resultados . . . 3
   2.1 Hallazgos transversales . . . 4
3. Consigna 1 — Testing unitario de `AsociadoDTO` . . . 4
   3.1 Requisitos de entrada . . . 4
   3.2 Contrato adoptado . . . 4
   3.3 Clases de equivalencia y valores límite . . . 5
   3.4 Batería de casos y valores límite . . . 5
   3.5 Trazabilidad de los casos . . . 5
   3.6 Resultados . . . 7
   3.7 Conclusiones . . . 7
4. Consigna 2 — Testing de integración . . . 7
   4.1 Componentes y requisitos . . . 7
   4.2 Estrategia de aislamiento . . . 7
   4.3 Escenarios de `BaseDeDatos` . . . 7
   4.4 Escenarios de `DataAccessObject` . . . 9
   4.5 Resultado de la ejecución . . . 10
   4.6 Correcciones aplicadas . . . 11
   4.7 Defectos documentados sin caso de prueba . . . 13
   4.8 Cobertura de los requisitos del SRS . . . 13
   4.9 Conclusiones . . . 14
5. Consigna 3 — Testing con mocks y concurrencia . . . 14
   5.1 Exigencias y componentes . . . 14
   5.2 Decisiones de diseño . . . 15
   5.3 Suite A — mocks sobre `State` . . . 15
   5.4 Suite B — concurrencia e integración . . . 17
   5.5 Resultados . . . 18
   5.6 Conclusiones . . . 19
6. Conclusiones generales . . . 19
7. Dificultades y limitaciones . . . 20
Apéndice A — Capturas de ejecución . . . 20
   A.1 Suite en rojo contra el código provisto (consignas 1 y 2) . . . 20
   A.2 Suite en verde tras las correcciones (consignas 1 y 2) . . . 20
   A.3 Suite en rojo de la consigna 3 . . . 21
   A.4 Cómo se reproduce la captura roja de las consignas 1 y 2 . . . 23
Apéndice B — Resultados de las corridas . . . 23

1. Introducción

El trabajo pide demostrar la capacidad de testear un producto de software estructurando las validaciones en **dos niveles arquitectónicos diferenciados**: el testing atómico (unitario) sobre componentes independientes y el testing de integración sobre componentes acoplados a recursos externos. A eso se suma una tercera consigna sobre dobles de prueba y concurrencia.

El objeto bajo prueba es la aplicación **Clínica**, provista por la cátedra. El requisito funcional que gobierna las consignas 1 y 2 es `SRS_Persistencia_Clinica.pdf`, y el enunciado es `Enunciado_Trabajo_final.pdf`.

1.1 Entorno

Componente	Versión	Nota
Java	1.8.0_504 (OpenJDK 64-Bit Server VM)	El proyecto declara `maven.compiler.source/target = 8`
Maven	3.10.0	—
JUnit	4.13.2	Técnica de la Semana 3
Mockito	4.11.0	Dobles de prueba de la consigna 3
Surefire	3.2.5	Ejecutor de las pruebas; reporta el detalle de cada caso
MariaDB	13.0.2-MariaDB (cliente 15.2)	Motor real de la consigna 2
mysql-connector-java	8.0.33	Driver JDBC; `BaseDeDatos` lo carga con `Class.forName`

Las versiones son las del entorno en que se ejecutaron las corridas que se documentan en §4, §5 y en el Apéndice B.

1.2 Cómo se ejecuta la suite

```
# desde la raíz del repositorio
mvn -f TPE2026/pom.xml test
```

La suite completa tiene **73 casos**. Las consignas 1 y 2 quedan en verde; la consigna 3 queda en rojo **a propósito**, porque solo demuestra detección (ver §5.2). Para correr únicamente las consignas 1 y 2:

```
mvn -f TPE2026/pom.xml test -Dtest=AsociadoDTOTest,BaseDeDatosTest,DataAccessObjectTest
```

2. Resumen de resultados

Consigna	Técnica	Casos	Rojo contra el código provisto	Estado final
1	Unitaria, caja negra aislada	23	15	**verde** (corregido)
2	Integración contra MariaDB real	34	17	**verde** (corregido)
3	Mocks y concurrencia	16	4	**rojo** (solo detección)
Total		**73**	**36**	

Los 36 resultados en rojo se corresponden con **23 defectos distintos** del código provisto, cada uno de ellos detectado por al menos un caso de prueba: los defectos de validación del DTO (**D1**, **D2**), los de la capa de persistencia (**D6**, **D7**, **D13**) y los de la máquina de estados y la rutina de retorno automático (**D15**, **D17**, **D18**, **D19**), más los catorce defectos de precondición del módulo de persistencia (**D23** a **D36**). Un mismo defecto puede disparar varios casos: la familia de precondiciones que el código provisto validaba con `assert` explica quince de los casos en rojo de la consigna 2, los catorce de **D23**–**D36** más el borrado por DNI de **D13**.

Los catorce defectos **D23**–**D36** corresponden, en realidad, a **doce validaciones distintas**: la del límite de la carga acotada y la del identificador de la carga individual son detectadas cada una por dos casos, el del valor interior y el del valor límite.

De los 36 resultados en rojo, 32 corresponden a las consignas 1 y 2, que la consigna 2 autoriza corregir, y 4 a la consigna 3, que solo pide demostrar la detección.

El inventario de defectos llega a **36 hallazgos numerados**. Dos de ellos eran errores de compilación y se corrigieron como prerrequisito; los que tienen un caso de prueba se corrigen junto con cada consigna, y el resto —no observable desde la interfaz pública— se documenta por inspección en lugar de tener un caso de prueba (§4.7 y §5.2).

2.1 Hallazgos transversales

Tres hallazgos afectan al proyecto entero y no a una consigna en particular:

1. **El proyecto entregado no compilaba.** `mvn test` fallaba antes de ejecutar un solo test, por dos errores en el paquete `ambulancia`: `EnTallerState.java` declaraba `package model.modeloDominio.ambulancia;` (sin la `o`, fuera del paquete real) y el archivo `TrasladandoPacienteClinicasState.java` contenía `public class TrasladandoPacienteClinicaState` (nombre de archivo ≠ nombre de clase pública). Se corrigieron como prerrequisito; toda la evidencia de este informe es posterior a esa corrección.
2. **Las validaciones de integridad estaban implementadas con `assert`.** Un `assert` es un auxilio de depuración: la JVM lo desactiva por defecto y lanza `AssertionError`, que es un `Error` y no una `Exception`. El SRS habla de *"validaciones de integridad"*, que deben estar siempre activas. Este patrón está extendido en el proyecto. Dentro del módulo de persistencia, las trece verificaciones de precondición que el código provisto hacía con `assert` quedaron convertidas en verificaciones explícitas que lanzan `IllegalArgumentException` (§4.6); las que quedan en el resto del proyecto (`modelo/`, `vista/`) están fuera de los requisitos del SRS y se documentan como observación.
3. **La contraseña y la URL de la base están hardcodeadas** en `BaseDeDatos` (`progra_c` / `progra_c`, base `Grupo_7`). No es un defecto de comportamiento, pero condiciona cómo se ejecuta la consigna 2.

---

3. Consigna 1 — Testing unitario de `AsociadoDTO`

3.1 Requisitos de entrada

Del SRS §3.1, `AsociadoDTO` es un contenedor de datos con reglas estructurales:

Atributo	Tipo	Regla
`nombre`, `apellido`, `dni`, `ciudad`, `calle`, `telefono`	`String`	No pueden ser nulos
`numero`	`int`	Debe ser mayor o igual a 0
(`dni`)	—	Actúa como identificador único

3.2 Contrato adoptado

Un dato inválido se rechaza lanzando `IllegalArgumentException`.

El código provisto validaba con `assert`, que lanza `AssertionError` y desaparece sin `-ea`. Se adoptó entonces el contrato de *excepción* como el esperado, que es el que sostiene el SRS al hablar de validaciones de integridad. `Teoria/Semana 3` distingue explícitamente "asertos y excepciones" como conceptos separados, y `Teoria/Semana 6` enseña a reportar fallos con excepciones específicas.

Tres decisiones complementarias:

- **`numero = 0` es válido**, porque el SRS dice `>= 0`. El código exigía `> 0`.

- **La cadena vacía es inválida**, siguiendo el precedente de la propia práctica de caja negra de la cátedra (`Teoria/Semana 4`), cuyos casos 4.2–4.35 tratan `""` como dato inválido.
- **El constructor parametrizado quedó fuera de alcance**, porque la consigna acota las validaciones a *"las implementadas en los setters"*. Su falta de validación se documenta como observación, sin caso de prueba.

3.3 Clases de equivalencia y valores límite

Categoría	Clases Correctas	Clases Erróneas
Atributos `String`	Cadena no nula y no vacía **(1)**	Cadena nula **(2)** · Cadena vacía **(3)**
Atributo `numero`	`numero >= 0` **(4)**	`numero < 0` **(5)**

Por qué no hay combinaciones cruzadas. En la plantilla de la cátedra, `agregarAlumno` necesita 35 sub-casos porque validaba el objeto completo. En `AsociadoDTO` cada setter valida **solo su propio valor** y no existe ningún invariante entre atributos: la unicidad del `dni` es una restricción **entre registros**, no dentro del objeto. Las particiones por atributo son entonces exhaustivas, y el producto cartesiano solo agregaría casos redundantes.

3.4 Batería de casos y valores límite

Clases correctas

Clase	Entradas	Salida esperada	Valores límite
1	`setNombre("Juan")` … `setTelefono("2235551234")` (6 casos)	El valor se asigna y se recupera	El SRS no fija longitud ni formato
4	`setNumero(0)`, `setNumero(1)`, `setNumero(Integer.MAX_VALUE)`	El valor se asigna y se recupera	0 = límite inferior del dominio, dentro; 1 = primer positivo; MAX = extremo superior

Clases erróneas

Clase	Entradas	Salida esperada	Valores límite
2	`null` en cada uno de los 6 setters	`IllegalArgumentException`	`null` = frontera del dominio "no nulo"
3	`""` en cada uno de los 6 setters	`IllegalArgumentException`	`""` = frontera entre vacío y no vacío
5	`setNumero(-1)`, `setNumero(Integer.MIN_VALUE)`	`IllegalArgumentException`	−1 = justo fuera del dominio `>= 0`; MIN = extremo inferior

Total: 23 casos (9 correctos + 14 erróneos).

Se usa la **Opción 2** de la cátedra para las excepciones esperadas (`try-catch` + `fail`, frente a `@Test(expected = …)`), porque esta última no admite mensaje propio y el fallo quedaría sin la regla que se estaba verificando. La verificación se centraliza en un verificador compartido que además intercepta la excepción inesperada.

3.5 Trazabilidad de los casos

Cada caso lleva el identificador de la clase de equivalencia que cubre en el nombre del método de prueba, con el patrón `test<operación>_<entrada>_Clase<n>` (por ejemplo, `testSetNumero_LimiteCero_Clase4`). La correspondencia entre la tabla de particiones, la batería de la §3.4 y el método ejecutado es unívoca: hay 23 casos y 23 métodos.

Caso	Entrada	Método de prueba	Resultado contra el código provisto
Clase 1 — `nombre`	`setNombre("Juan")`	`testAsignacionYRecuperacion_Nombre_Clase1`	pass

Caso	Entrada	Método de prueba	Resultado contra el código provisto
Clase 1 — `apellido`	`setApellido("Perez")`	`testAsignacionY Recuperacion_ Apellido_Clase1`	pass
Clase 1 — `dni`	`setDni("12345678")`	`testAsignacionY Recuperacion_Dni_ Clase1`	pass
Clase 1 — `ciudad`	`setCiudad("Mar del Plata")`	`testAsignacionY Recuperacion_Ciudad_ Clase1`	pass
Clase 1 — `calle`	`setCalle("Belgrano")`	`testAsignacionY Recuperacion_Calle_ Clase1`	pass
Clase 1 — `telefono`	`setTelefono( "2235551234")`	`testAsignacionY Recuperacion_ Telefono_Clase1`	pass
Clase 2 — `nombre`	`setNombre(null)`	`testSetNombre_Nulo_ Clase2`	fail
Clase 2 — `apellido`	`setApellido(null)`	`testSetApellido_ Nulo_Clase2`	fail
Clase 2 — `dni`	`setDni(null)`	`testSetDni_Nulo_ Clase2`	fail
Clase 2 — `ciudad`	`setCiudad(null)`	`testSetCiudad_Nulo_ Clase2`	fail
Clase 2 — `calle`	`setCalle(null)`	`testSetCalle_Nulo_ Clase2`	fail
Clase 2 — `telefono`	`setTelefono(null)`	`testSetTelefono_ Nulo_Clase2`	fail
Clase 3 — `nombre`	`setNombre("")`	`testSetNombre_Vacio_ Clase3`	fail
Clase 3 — `apellido`	`setApellido("")`	`testSetApellido_ Vacio_Clase3`	fail
Clase 3 — `dni`	`setDni("")`	`testSetDni_Vacio_ Clase3`	fail
Clase 3 — `ciudad`	`setCiudad("")`	`testSetCiudad_Vacio_ Clase3`	fail
Clase 3 — `calle`	`setCalle("")`	`testSetCalle_Vacio_ Clase3`	fail
Clase 3 — `telefono`	`setTelefono("")`	`testSetTelefono_ Vacio_Clase3`	fail
Clase 4 — `numero = 0`	`setNumero(0)`	`testSetNumero_Limite Cero_Clase4`	fail
Clase 4 — `numero = 1`	`setNumero(1)`	`testSetNumero_ Positivo_Clase4`	pass
Clase 4 — `numero = MAX`	`setNumero(Integer. MAX_VALUE)`	`testSetNumero_ Maximo_Clase4`	pass
Clase 5 — `numero = -1`	`setNumero(-1)`	`testSetNumero_ NegativoUno_Clase5`	fail
Clase 5 — `numero = MIN`	`setNumero(Integer. MIN_VALUE)`	`testSetNumero_ Minimo_Clase5`	fail

3.6 Resultados

Contra el código provisto:

```
Tests run: 23, Failures: 15, Errors: 0, Skipped: 0
```

Las 15 fallas se reparten como se predijo: las 6 de la clase 2 y las 2 de la clase 5 lanzaban `AssertionError` en lugar de `IllegalArgumentException`, las 6 de la clase 3 no lanzaban nada (el `assert != null` acepta `""`), y el valor límite `0` fallaba contra el `assert numero > 0`. Los quince casos en rojo se registran como fallas de JUnit, no como errores.

Tras la corrección del código (autorizada por la consigna 2):

```
Tests run: 23, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Ciclo rojo → verde completo, sin modificar ningún test.

3.7 Conclusiones

- La suite **detecta los tres defectos** que el código provisto tenía en este componente: el mecanismo de validación equivocado, el agujero de la cadena vacía y la contradicción con el SRS en el límite `0`.
- La caja negra con particiones y valores límite **alcanzó con 23 casos**. La reducción frente a los 35 del ejemplo de la cátedra no es un atajo: se justifica en que aquí no hay invariantes entre atributos.
- El valor límite `0` fue el caso más valioso: es el único que detecta una **contradicción entre el código y el requisito**, y no un error de implementación. Sin el análisis de valores límite, ese defecto pasaba inadvertido.
- El hecho de que las validaciones fueran `assert` tiene una consecuencia de producción que excede al test: con la JVM por defecto **no validaban nada**.

---

4. Consigna 2 — Testing de integración

4.1 Componentes y requisitos

Componente	Requisitos del SRS
`BaseDeDatos`	Singleton de instancia única; conexión con parámetros predefinidos; `iniciarBD()` crea el esquema, recrea `Asociados` y la puebla; operaciones con `PreparedStatement`
`DataAccessObject`	Guardado con validación de nulo; carga individual, en lotes acotados y completa; actualización y borrado por **criterio único (DNI)**
`AsociadoDTO`	Participa en el mapeo de ida y vuelta

4.2 Estrategia de aislamiento

`iniciarBD()` es `public` y ejecuta `DROP TABLE` + `CREATE TABLE` + la inserción de 20 registros semilla. Cada test lo invoca desde `@Before`, de modo que todos parten del mismo estado conocido y el orden de ejecución no importa. La semilla tiene 20 filas; los escenarios usan las filas de `dni` `12345678` (id 1) y `11223344` (id 3), **dos filas distintas a propósito**, para poder verificar que una operación sobre una no afecta a la otra.

Las verificaciones usan **JDBC directo** sobre `getConexion()` como oráculo independiente, sin apoyarse en los métodos bajo prueba.

4.3 Escenarios de `BaseDeDatos`

La columna **Resultado** registra lo observado contra el código provisto: `pass` en verde, `fail` en rojo. Los casos en `fail` no son errores de la prueba: son la detección de los defectos que el SRS exige verificar (§4.6).

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
I1	Instancia única	`getInstancia()` invocado dos veces	Ambas invocaciones devuelven la misma referencia	`testGetInstanciaDevuelveLaMismaReferencia_I1`	pass
I2	Conexión disponible	`getConexion()` después de `iniciarBD()`	La conexión no es nula	`testConexionNoEsNula_I2`	pass
I3	Esquema y semilla	`iniciarBD()`	La tabla `Asociados` queda creada y poblada con 20 filas	`testIniciarBDEstableceLaSemilla_I3`	pass
I4	Recreación idempotente	`iniciarBD()` invocado dos veces seguidas	La tabla sigue con 20 filas: no acumula	`testIniciarBDEsIdempotente_I4`	pass
I5	Consulta parametrizada	`ejecutarConsulta("SELECT * FROM Asociados WHERE id = ? ;", 1)`	Devuelve la fila del id 1: `dni` = `12345678`, `nombre` = `Juan`	`testEjecutarConsultaConParametroDevuelveLaFilaIndicada_I5`	pass
I6	Error del motor en la consulta	`ejecutarConsulta("SELECT * FROM TablaInexistente ;", -1)`	Lanza `SQLException` con mensaje	`testEjecutarConsultaSobreTablaInexistenteLanzaSQLException_I6`	pass
I7	Inserción parametrizada	`ejecutarInsert` con un DTO de DNI `90000001`	La tabla pasa a 21 filas y el `nombre` del DNI insertado es `Nuevo`	`testEjecutarInsertIncorporaElRegistro_I7`	pass
I8	DNI duplicado en la inserción	`ejecutarInsert` con un DTO de DNI `12345678`, ya presente	Lanza `SQLException` por la restricción `UNIQUE`; la tabla no cambia	`testEjecutarInsertConDniDuplicadoLanzaSQLException_I8`	pass
I9	Borrado parametrizado	`ejecutarDelete("DELETE FROM Asociados WHERE dni = ? ;", "12345678")`	La tabla pasa a 19 filas	`testEjecutarDeleteEliminaElRegistroIndicado_I9`	pass
I10	Borrado sin coincidencia	`ejecutarDelete("DELETE FROM Asociados WHERE dni = ? ;", "00000000")`	La tabla sigue con 20 filas	`testEjecutarDeleteConDniInexistenteNoEliminaNada_I10`	pass
I11	El DNI viaja como dato	`ejecutarDelete("DELETE FROM Asociados WHERE dni = ? ;", "' OR '1'='1")`	La tabla sigue con 20 filas: el valor es parámetro, no SQL concatenado	`testEjecutarDeleteTrataElDniComoDatoYNoComoSql_I11`	pass
I27	Consulta nula	`ejecutarConsulta(null, 1)`	`IllegalArgumentException`; la tabla no cambia	`testEjecutarConsultaConSqlNuloEsRechazado_I27`	**fail**

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
I28	Sentencia de actualización nula	`ejecutar Actualizacion( null, DTO)`	`IllegalArgument Exception`; la tabla no cambia	`testEjecutar ActualizacionCon SqlNuloEs Rechazado_I28`	**fail**
I29	Sentencia de inserción nula	`ejecutarInsert( null, DTO)`	`IllegalArgument Exception`; la tabla no cambia	`testEjecutar InsertConSqlNulo EsRechazado_I29`	**fail**
I30	Sentencia de borrado nula	`ejecutarDelete( null, "12345678")`	`IllegalArgument Exception`; la tabla no cambia	`testEjecutar DeleteConSqlNulo EsRechazado_I30`	**fail**
I31	DNI nulo en el borrado	`ejecutarDelete( "DELETE FROM Asociados WHERE dni = ? ;", null)`	`IllegalArgument Exception`; la tabla no cambia	`testEjecutar DeleteConDniNulo EsRechazado_I31`	**fail**
I33	Asociado nulo en la actualización	`ejecutar Actualizacion( "UPDATE Asociados SET … WHERE dni = ? ;", null)`	`IllegalArgument Exception`; la tabla no cambia	`testEjecutar ActualizacionCon AsociadoNuloEs Rechazado_I33`	**fail**
I34	Asociado nulo en la inserción	`ejecutarInsert( "INSERT INTO Asociados(…) VALUES (…);", null)`	`IllegalArgument Exception`; la tabla no cambia	`testEjecutar InsertCon AsociadoNuloEs Rechazado_I34`	**fail**

18 casos: 11 en `pass` y 7 en `fail`.

4.4 Escenarios de `DataAccessObject`

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
I12	Guardado de un asociado	`guardar( Asociado)` con DNI `90000001`	La tabla pasa a 21 filas y el registro es recuperable por su DNI	`testGuardar IncorporaEl Asociado_I12`	pass
I13	Guardado con DNI duplicado	`guardar( Asociado)` con DNI `12345678`, ya presente	Lanza `SQLException` por la restricción `UNIQUE`; la tabla no cambia	`testGuardarCon DniDuplicado LanzaSQL Exception_I13`	pass
I14	Carga completa	`cargarBD()`	Devuelve los 20 DTO de la semilla	`testCargarB DevuelveTodaLa Semilla_I14`	pass
I15	Carga acotada	`cargarConLimite( 5)`	Devuelve exactamente 5 DTO	`testCargarCon LimiteDevuelveLa CantidadPedida_ I15`	pass

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
I16	Carga individual	`cargar(3)`	Devuelve el DTO de **esa** fila: `dni` = `11223344`, `nombre` = `Carlos`	`testCargarDevuelveElAsociadoIndicado_I16`	**fail**
I17	Actualización por criterio único	`actualizar(Asociado)` con DNI `11223344`	Cambia la fila de ese DNI (`nombre` → `Nombre Actualizado`) y no toca la del DNI `12345678`	`testActualizarModificaLaFilaDelDniIndicado_I17`	**fail**
I18	Eliminación por DNI	`eliminar("12345678")`	La tabla pasa a 19 filas	`testEliminarQuitaElAsociadoIndicado_I18`	pass
I19	Eliminación con DNI nulo	`eliminar(null)`	`IllegalArgumentException`	`testEliminarConDniNuloEsRechazado_I19`	**fail**
I20	Guardado con asociado nulo	`guardar(null)`	`IllegalArgumentException`; la tabla no cambia	`testGuardarConAsociadoNuloEsRechazado_I20`	**fail**
I21	Actualización con asociado nulo	`actualizar(null)`	`IllegalArgumentException`; la tabla no cambia	`testActualizarConAsociadoNuloEsRechazado_I21`	**fail**
I22	Límite de carga igual a cero	`cargarConLimite(0)`	`IllegalArgumentException`; la tabla no cambia	`testCargarConLimiteCeroEsRechazado_I22`	**fail**
I23	Límite de carga negativo	`cargarConLimite(-1)`	`IllegalArgumentException`; la tabla no cambia	`testCargarConLimiteNegativoEsRechazado_I23`	**fail**
I24	Identificador igual a cero	`cargar(0)`	`IllegalArgumentException`; la tabla no cambia	`testCargarConIdCeroEsRechazado_I24`	**fail**
I25	Identificador negativo	`cargar(-1)`	`IllegalArgumentException`; la tabla no cambia	`testCargarConIdNegativoEsRechazado_I25`	**fail**
I26	Mapeo con asociado nulo	`CrearDTO((Asociado) null)`	`IllegalArgumentException`; la tabla no cambia	`testCrearDTOConAsociadoNuloEsRechazado_I26`	**fail**
I32	Límite de longitud del DNI	`guardar(Asociado)` con `dni` = `12345678901` (11 caracteres)	`SQLException` por el límite del esquema (`DNI VARCHAR(10)`); la tabla no cambia	`testGuardarConDniDeOnceCaracteresLanzaSQLException_I32`	pass

16 casos: 6 en `pass` y 10 en `fail`.

El caso **I32** es un valor límite del esquema, no un defecto: el motor rechaza el DNI de once caracteres con el error de truncamiento de la columna, que es exactamente la excepción que el caso espera. Por eso queda en `pass` incluso contra el código provisto.

4.5 Resultado de la ejecución

Las tres suites del módulo de persistencia se ejecutan juntas, desde la raíz del proyecto, con:

```
mvn -f TPE2026/pom.xml test -Dtest=AsociadoDTOTest,BaseDeDatosTest,DataAccessObjectTest
```

La salida de esa corrida termina en:

```
Results:
Tests run: 57, Failures: 30, Errors: 2, Skipped: 0
BUILD FAILURE
```

Suite	Casos	En rojo	En verde
`AsociadoDTOTest` (consigna 1)	23	15	8
`BaseDeDatosTest` (consigna 2)	18	7	11
`DataAccessObjectTest` (consigna 2)	16	10	6
Total consigna 2	**34**	**17**	**17**
Total consignas 1 y 2	**57**	**32**	**25**

De los 32 casos en rojo, 30 se cuentan como fallas y 2 como errores de JUnit: son **I16** e **I17**, donde la excepción del motor llega sin que el caso la espere, y por eso JUnit los registra como errores y no como fallas.

Tras las correcciones de §4.6, la misma corrida da `Tests run: 57, Failures: 0, Errors: 0, Skipped: 0` y `BUILD SUCCESS`: los 34 casos de esta consigna quedan en `pass` sin modificar ningún test, porque la corrección estuvo en el código de producción y no en las aserciones.

4.6 Correcciones aplicadas

Las correcciones de esta sección están autorizadas por la consigna 2, que permite corregir el código del ítem 1 y el propio módulo de persistencia. Cada defecto se documenta con qué se rompía, dónde, cómo lo detectó el caso de prueba y cómo se corrigió.

Defecto	Caso	Qué se rompía	Traza observada	Corrección
D6	I16	La carga individual leía la fila antes de avanzar el cursor (`DataAccessObject.cargar(int)`)	`java.sql.SQLException: Before start of result set`	Se llama a `resultado.next()` antes de leer y se devuelve `null` cuando no hay coincidencia
D7	I17	La actualización no usaba el criterio único del SRS: fijaba el parámetro del identificador en `1` y escribía siempre sobre la primera fila (`BaseDeDatos.ejecutarActualizacion`)	`java.sql.SQLIntegrityConstraintViolationException: Duplicate entry '11223344' for key 'DNI'`	Se adopta el **DNI** como criterio único (SRS §3.3): `UPDATE … WHERE dni = ?` y el parámetro 8 enlazado al DNI del DTO
D13	I19	El borrado por DNI no verificaba su precondición: sólo rechazaba la cadena vacía y confundía el vacío con el nulo (`DataAccessObject.eliminar`)	`Se esperaba IllegalArgumentException para un DNI nulo, pero se lanzo java.lang.AssertionError`	Verificación explícita que lanza `IllegalArgumentException` cuando el DNI es `null` o vacío
D23	I20	La precondición de nulidad del guardado se validaba con `assert`, que la JVM desactiva por defecto (`DataAccessObject.guardar`)	`AssertionError: El asociado no puede ser nulo` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`

Defecto	Caso	Qué se rompía	Traza observada	Corrección
D24	I21	La precondición de nulidad de la actualización se validaba con `assert` (`DataAccessObject.actualizar`)	`AssertionError: El asociado no puede ser nulo` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D25	I22	El límite de la carga acotada se validaba con `assert limite > 0` (`DataAccessObject.cargarConLimite`)	`AssertionError: El limite debe ser mayor que 0` al pedir un lote de 0 registros	Verificación explícita que lanza `IllegalArgumentException`
D26	I23	El mismo `assert limite > 0`, en su valor límite: un lote negativo (`DataAccessObject.cargarConLimite`)	`AssertionError: El limite debe ser mayor que 0` al pedir un lote negativo	Verificación explícita que lanza `IllegalArgumentException`
D27	I24	El identificador de la carga individual se validaba con `assert id > 0` (`DataAccessObject.cargar`)	`AssertionError: El id debe ser mayor que 0` con identificador 0	Verificación explícita que lanza `IllegalArgumentException`
D28	I25	El mismo `assert id > 0`, en su valor límite: un identificador negativo (`DataAccessObject.cargar`)	`AssertionError: El id debe ser mayor que 0` con identificador negativo	Verificación explícita que lanza `IllegalArgumentException`
D29	I26	La precondición de nulidad del mapeo a DTO se validaba con `assert` (`DataAccessObject.CrearDTO`)	`AssertionError: El asociado no puede ser nulo` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D30	I27	La consulta nula se validaba con `assert consulta != null` (`BaseDeDatos.ejecutarConsulta`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D31	I28	La sentencia de actualización nula se validaba con `assert actualizacion != null` (`BaseDeDatos.ejecutarActualizacion`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D32	I29	La sentencia de inserción nula se validaba con `assert insert != null` (`BaseDeDatos.ejecutarInsert`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D33	I30	La sentencia de borrado nula se validaba con `assert delete != null` (`BaseDeDatos.ejecutarDelete`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D34	I31	El DNI nulo se validaba con `assert dni != null` (`BaseDeDatos.ejecutarDelete`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`
D35	I33	El asociado nulo en la actualización se validaba con `assert aDTO != null` (`BaseDeDatos.ejecutarActualizacion`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`

Defecto	Caso	Qué se rompía	Traza observada	Corrección
D36	I34	El asociado nulo en la inserción se validaba con `assert a DTO != null` (`BaseDeDatos.ejecutarInsert`)	`AssertionError` en lugar de `IllegalArgumentException`	Verificación explícita que lanza `IllegalArgumentException`

Los catorce defectos **D23**–**D36** corresponden a **doce validaciones** del módulo de persistencia: la del límite de la carga acotada (**D25** y **D26**) y la del identificador de la carga individual (**D27** y **D28**) son la misma validación detectada por dos casos, el del valor interior y el del valor límite.

Alcance de la corrección. En el módulo de persistencia **no queda ninguna precondición validada con `assert`**: las trece verificaciones del código provisto —doce de ellas en **D23**–**D36** y la del borrado por DNI, corregida con **D13**— son ahora verificaciones explícitas que lanzan `IllegalArgumentException`. Ninguna otra línea de `BaseDeDatos.java` ni de `DataAccessObject.java` fue modificada. Las validaciones con `assert` que quedan en el resto del proyecto (`modelo/`, `vista/`) están fuera de los requisitos del SRS §3.1–§3.3 y se documentan como observación.

4.7 Defectos documentados sin caso de prueba

No todo defecto es observable desde la interfaz pública. Estos se documentan por inspección, y se explicita por qué no se les inventó una aserción:

Defecto	Por qué no hay test
D8 — `CrearDTO(int id, …)` recibe el `id` y lo descarta	El `AsociadoDTO` **no tiene** campo `id` ni getter: la pérdida no es observable desde la interfaz pública. Se manifiesta como la causa raíz de **D7**
D9 — `getInstancia()` no está sincronizado	La carrera solo puede ocurrir en la **primera** invocación. Para cuando el test corre, la instancia ya existe y siempre devolvería la misma referencia: el test pasaría siempre y daría falsa confianza
D10 — `abrirConexion()` traga `SQLException` con `printStackTrace`	Requeriría que la conexión fallara **antes** de que el Singleton se inicialice; `URL` y credenciales están hardcodeados (`private static final`), así que no hay forma de inyectar una configuración inválida desde el test
D12 — `ejecutarConsulta` no cierra el `PreparedStatement`	Es una fuga de recursos, no un cambio de comportamiento observable; se detecta por inspección del código (el `close()` está comentado)

Diferidos, por la misma razón:

- **`cerrarConexion()`**: no se prueba. Cierra la conexión del Singleton compartido y dejaría inválidos todos los tests siguientes; el orden de ejecución no está garantizado. Se verifica por inspección.
- **`Domicilio` con `numero = 0`**: la clase de dominio repite el `assert numero > 0` y bloquearía el camino `Asociado → Domicilio`. Queda como observación de propagación de **D2**, fuera del alcance del SRS del módulo de persistencia.

En el módulo de persistencia no queda ningún defecto de precondición sin caso de prueba: la familia de validaciones con `assert` está cerrada y cada miembro tiene su caso (**D23**–**D36** y **D13**).

4.8 Cobertura de los requisitos del SRS

Cada requisito del SRS §3.2 y §3.3 está cubierto por al menos un caso de prueba:

Requisito	Casos que lo cubren
§3.2 — Instancia única del manejador de conexión (Singleton)	I1

Requisito	Casos que lo cubren
§3.2 — Conexión inicializada con parámetros de configuración predefinidos	I2
§3.2 — Rutina de configuración del entorno: crear el esquema, eliminar y recrear `Asociados`, poblarla con registros de prueba	I3, I4
§3.2 — Interfaces de ejecución parametrizadas con `PreparedStatement` (consultas, inserciones, actualizaciones y eliminaciones)	I5, I6, I7, I8, I9, I10, I11, I27, I28, I29, I30, I31, I33, I34
§3.3 — Guardado: validar que la entidad no sea nula, convertirla a DTO y ejecutar la inserción	I12, I13, I20, I32
§3.3 — Carga individual por identificador único	I16, I24, I25
§3.3 — Carga en lotes acotados por un límite especificado	I15, I22, I23
§3.3 — Carga como colección completa	I14
§3.3 — Actualización y eliminación por criterio único (DNI)	I17, I18, I19, I21, I26

Los casos I6, I8 e I13 cubren además la detección de errores del motor que el SRS anticipa (*"uso incorrecto o inconsistente de archivos o bases de datos"*), y el caso I11 descarta la inyección de SQL al comprobar que el DNI viaja como parámetro y no concatenado.

4.9 Conclusiones

- La suite de integración **detecta los defectos** del módulo de persistencia: el error de mapeo por posición del cursor, el criterio de actualización incorrecto, la precondición sin verificar y las doce validaciones de precondición que el código provisto dejaba desactivables.
- La ampliación de la batería a **34 escenarios** no agrandó el alcance de la corrección: los catorce casos nuevos detectaron doce validaciones de precondición que el código provisto dejaba desactivables, y la corrección se hizo con una verificación explícita por caso, sin reescribir el módulo.
- El escenario de **excepciones SQL** funcionó en los dos sentidos previstos por la cátedra: se verifican errores del motor (tabla inexistente, DNI duplicado) y se verifica que el borrado viaje como **parámetro** y no concatenado, lo que descarta la inyección de SQL.
- El defecto de actualización fue el de mayor valor didáctico: solo aparece si el caso de prueba **no usa la fila 1**. Un test que hubiera actualizado esa fila habría pasado, y el defecto habría quedado oculto.
- El valor límite del esquema (I32) se distingue de un defecto: el motor rechaza el DNI de once caracteres con la excepción que el caso espera, y esa diferencia de criterio es la que separa "el sistema se comporta como pide el SRS" de "el sistema tiene un defecto".

---

5. Consigna 3 — Testing con mocks y concurrencia

5.1 Exigencias y componentes

La consigna pide dos cosas: pruebas **atómicas con dobles de prueba** sobre el diseño desacoplado de la interfaz `State`, simulando estados "ante la ausencia temporal de las implementaciones concretas"; y pruebas de **integración y concurrencia** sobre la máquina de estados acoplada a `Ambulancia`, validando recursos compartidos, bloques sincronizados e hilos independientes.

`Ambulancia` es `Observable` y mantiene el estado actual, el flag `ocupado`, el flag de simulación activa y una lista de asociados. Sus operaciones son `synchronized`. El punto de inyección para los mocks es `setState(State)`, que es público.

5.2 Decisiones de diseño

Alcance: solo detección. La consigna pide *"demostrar… que la suite descubre los problemas"*, a diferencia de la consigna 2, que sí autoriza corregir. Por eso **no se modificó código de producción** y los cuatro defectos quedan en rojo como evidencia.

No se usaron hilos reales de `Asociado` u `Operario`. `TiempoMuerto.esperar()` duerme un valor aleatorio de hasta **8 segundos** entre servicios, así que una prueba con esos hilos sería lenta y no determinista. La concurrencia se ejercita directamente sobre los métodos `synchronized` de `Ambulancia`, con hilos propios coordinados por `CountDownLatch`.

No se modificó el intervalo de `RetornoAutomatico`. Es `private static final = 10000` y no tiene costura de inyección. Se descartó bajarlo por reflexión (frágil) y omitir el test (la consigna nombra esa clase explícitamente). El caso correspondiente usa un `timeout` de 12 s.

5.3 Suite A — mocks sobre `State`

Condiciones generales de entrada, comunes a los casos de esta suite: la `Ambulancia` se construye con su constructor por defecto, es decir con la simulación activa, sin asociados en la lista y con el estado `Disponible` `State`; el `State` inyectado es un doble de prueba que no contiene comportamiento propio.

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
M1	Delegación de la atención a domicilio	`atenderDomicilio(asociado())` con un `State` simulado inyectado por `setState` y un `Asociado` simulado	Invoca `State.solicitudDeAtencionDomicilio()` exactamente una vez	`testAtenderDomicilioDelegaEnElEstado_M1`	pass
M2	Delegación del traslado a la clínica	`trasladarALaClinica(asociado())` con un `State` simulado inyectado por `setState` y un `Asociado` simulado	Invoca `State.solicitudDeTraslado()` exactamente una vez	`testTrasladarALaClinicaDelegaEnElEstado_M2`	pass
M3	Delegación de la solicitud de mantenimiento	`solicitarMantenimiento(operario())` con un `State` simulado inyectado por `setState` y un `Operario` simulado	Invoca `State.solicitudMantenimiento()` exactamente una vez	`testSolicitarMantenimientoDelegaEnElEstado_M3`	pass
M4	Delegación del retorno a la clínica	`regresarSinPaciente()` con un `State` simulado inyectado por `setState`	Invoca `State.retornoClinica()` exactamente una vez	`testRegresarSinPacienteDelegaEnElEstado_M4`	pass

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
M5	La atención a domicilio ocupa la ambulancia	`atenderDomicilio(asociado())` con un `State` simulado inyectado por `setState`	`isOcupado()` queda en `true`	`testAtenderDomicilioMarcaLaAmbulanciaOcupada_M5`	pass
M6	La atención a domicilio no procede con la simulación inactiva	`setSimulacionActiva(false)` y luego `atenderDomicilio(asociado())`, con un `State` simulado inyectado por `setState`	No invoca `State.SolicitudDeAtencionDomicilio()` y `isOcupado()` queda en `false`	`testAtenderDomicilioNoDelegaSiLaSimulacionNoEstaActiva_M6`	pass
M7	El retorno automático no actúa sobre una ambulancia ocupada	Estado `AtendiendoEnDomicilioState` y `setOcupado(true)`; luego `retornoAutomatico(retorno)`, con un `RetornoAutomatico` construido sobre la misma ambulancia	`getEstadoActual()` sigue siendo la **misma instancia** de `AtendiendoEnDomicilioState`	`testRetornoAutomaticoNoCambiaElEstadoSiLaAmbulanciaEstaOcupada_M7`	**fail**
M8	La simulación se desactiva cuando todos los asociados terminaron	Un `Asociado` simulado con `getCantSolicitudesAtendidas() = 3` y `getMaxCantSolicitudes() = 3` en la lista de asociados; luego `verificarYFinalizarSimulacion()`	`isSimulacionActiva()` pasa a `false`	`testVerificarYFinalizarSimulacionLaDesactivaSiTodosTerminaron_M8`	pass
M9	La simulación se mantiene con un asociado pendiente	Un `Asociado` simulado con `getCantSolicitudesAtendidas() = 1` y `getMaxCantSolicitudes() = 3` en la lista de asociados; luego `verificarYFinalizarSimulacion()`	`isSimulacionActiva()` sigue en `true`	`testVerificarYFinalizarSimulacionLaMantieneSiQuedaAlgunAsociado_M9`	pass

9 casos: 8 en `pass` y 1 en `fail`.

5.4 Suite B — concurrencia e integración

Los hilos se coordinan con `CountDownLatch` y se ejercitan directamente los métodos `synchronized` de la `Ambulancia`. Los casos que dependen de un estado concreto lo inyectan con `setState`, sin simulaciones: son los que verifican la transición de la máquina de estados.

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
C1	Exclusión mutua entre los métodos sincronizados	Un hilo entra a `atenderDomicilio(asociado())` y el `State` simulado retiene el monitor hasta que se lo libere; un segundo hilo invoca `isOcupado()`	El segundo hilo no puede entrar mientras el primero retiene el monitor, y entra al liberarse	`testLosMetodosSincronizadosSeExcluyenEntreHilos_C1`	pass
C2	Un hilo en espera se libera cuando termina la simulación	`setOcupado(true)`; un hilo invoca `atenderDomicilio(asociado())` y queda esperando; después `setSimulacionActiva(false)`	El hilo termina: el `notifyAll` de la simulación finalizada lo despierta y sale sin delegar	`testUnHiloBloqueadoSeLiberaCuandoTerminaLaSimulacion_C2`	pass
C3	Recorrido de la lista de asociados mientras se modifica	Un hilo invoca `agregarAsociado(asociado)` de forma continua y otro invoca `verificarYFinalizarSimulacion()` de forma continua, durante un intervalo acotado de 2 s	Ninguna de las dos operaciones lanza `ConcurrentModificationException`	`testAgregarAsociadosMientrasSeVerificaLaSimulacion_C3`	**fail**
C4	El retorno automático se dispara con la simulación activa	Una `Ambulancia` simulada con `isSimulacionActiva()` en `true`, y un hilo `daemon` que ejecuta `new RetornoAutomatico(ambulancia)`	El hilo invoca `retornoAutomatico(...)` dentro del intervalo de 12 s	`testRetornoAutomaticoInvocaElRetornoConLaSimulacionActiva_C4`	**fail**
C5	El retorno de una ambulancia que traslada un paciente	Estado `TrasladandoPacienteClinicaState` inyectado por `setState`; luego `retornoAutomatico(retorno)`	`getEstadoActual()` pasa a ser una instancia de `RegresandoClinicaSinPacienteState`	`testElRetornoDeTrasladarPacienteVuelveSinPaciente_C5`	**fail**

id	Escenario	Entrada / Condiciones de entrada	Salida esperada	Método de prueba	Resultado
C6	El traslado parte de la ambulancia disponible	Estado `DisponibleState` inyectado por `setState`; luego `trasladarALaClinica(asociado())`	`getEstadoActual()` pasa a ser una instancia de `TrasladandoPacienteClinicaState`	`testTrasladoDesdeDisponibleCambiaElEstado_C6`	pass
C7	El mantenimiento parte de la ambulancia disponible	Estado `DisponibleState` inyectado por `setState`; luego `solicitarMantenimiento(operario())`	`getEstadoActual()` pasa a ser una instancia de `EnTallerState` e `isOcupado()` queda en `true`	`testMantenimientoDesdeDisponibleOcupaLaAmbulancia_C7`	pass

7 casos: 4 en `pass` y 3 en `fail`.

5.5 Resultados

Los cuatro casos en rojo (M7, C3, C4 y C5) son intencionales. La consigna 3 pide demostrar que la suite descubre los problemas de sincronización y de estado, no corregir el código de producción: a diferencia de la consigna 2, no autoriza tocarlo. El rojo de esos cuatro casos **es el resultado esperado**, y es la evidencia de que la suite cumple lo que se le pedía.

La corrida de la suite completa termina en:

```
Tests run: 73, Failures: 4, Errors: 0, Skipped: 0
BUILD FAILURE
```

Las cuatro fallas y sus trazas:

Defecto	Caso	Traza
D15 — `RetornoAutomatico.run()` tiene la condición del bucle invertida	C4	el `verify` con `timeout(12000)` no se satisface: con la simulación activa la rutina retorna sin llamar nunca (`Wanted but not invoked: ambulancia.retornoAutomatico(...)`)
D17 — `retornoAutomatico` no verifica si el retorno es apropiado	M7	`expected same:<AtendiendoEnDomicilioState> was not:<RegresandoClinicaSinPacienteState>`
D18 — `TrasladandoPacienteClinicaState.RetornoClinica()` transiciona a un estado incorrecto	C5	`Debe volver a la clinica sin paciente, pero quedo en RegresandoDelTallerState`
D19 — la lista de asociados no está sincronizada	C3	`ConcurrentModificationException` al recorrerla mientras se la modifica

El primer defecto merece una aclaración: el bucle es `while (!ambulancia.isSimulacionActiva())`, con la condición **negada**. Como consecuencia, con la simulación activa el cuerpo nunca se ejecuta y la rutina de retorno automático **no retorna nunca** — que es exactamente lo contrario de lo que su propio comentario declara.

Y el tercero: la ambulancia está **trasladando un paciente a la clínica**, así que su retorno solo puede ser *"regresando a la clínica sin paciente"*, nunca *"regresando del taller"*. La misma transición en `AtendiendoEnDomicilioState` apunta al estado correcto, lo que confirma la lectura.

El caso M7 es el hallazgo del lado de los mocks: el propio código documenta la intención en `RetornoAutomatico.run()` (*"El método retornoAutomatico en Ambulancia debe verificar si es apropiado hacer el retorno según el estado actual"*), pero la implementación ignora la ocupación y fuerza la transición.

5.6 Conclusiones

- El diseño desacoplado de `State` **habilita el aislamiento**: inyectando un mock se verifica qué operación delega la ambulancia en cada caso, sin depender de las implementaciones concretas. Es el patrón que la cátedra muestra con `TestIntegracionMocks`.
- **El doble de prueba no solo sirve para aislar: también sirve para sincronizar.** En `C1` el mock de `State` retiene el monitor dentro de una operación `synchronized`, lo que permite verificar la exclusión mutua de forma determinista, sin depender de duraciones ni de `sleep`.
- Los dos defectos de concurrencia son de naturaleza distinta: `C3` es un **conflicto entre componentes** (una clase corriendo mientras otra modifica el mismo recurso), y el defecto de `RetornoAutomatico` es una **condición lógica invertida** que se manifiesta como un fallo de sincronización. Distinguirlos importa para el diagnóstico.
- `C3` es una condición de carrera y por eso se implementó con los dos hilos compitiendo durante un intervalo acotado, fallando en cuanto se observa la excepción. Es una prueba inherentemente probabilística, y así queda declarada.
- Los cuatro defectos **no se corrigieron**, por decisión de alcance. Quedan como evidencia de que la suite los descubre, que es lo que la consigna pide.

---

6. Conclusiones generales

1. **La separación de niveles se sostuvo.** Las 23 pruebas unitarias de la consigna 1 corren sin ningún servicio externo; las 34 de la consigna 2 exigen MariaDB real. Ninguna prueba unitaria toca la base, y ninguna prueba de integración valida la lógica interna del DTO.
2. **Cada técnica detectó lo que le corresponde.** La caja negra con valores límite encontró la contradicción entre código y requisito en `numero = 0`; la integración encontró los errores de mapeo y de criterio de actualización, y la familia completa de precondiciones desactivables; los mocks encontraron el incumplimiento de un contrato documentado, y la concurrencia encontró el conflicto por recurso compartido.
3. **La cobertura de los requisitos del SRS es total.** Cada requisito de §3.1, §3.2 y §3.3 tiene al menos un caso de prueba que lo verifica (§4.8), y el inventario de 57 casos de las consignas 1 y 2 no deja ningún requisito sin cubrir.
4. **La familia de validaciones con `assert` del módulo del SRS quedó convertida.** Las trece precondiciones que el código provisto validaba de forma desactivable son ahora verificaciones explícitas que lanzan `IllegalArgumentException`; en ese módulo no queda ninguna validación que la JVM pueda apagar. Las que quedan en el resto del proyecto quedan documentadas como observación.
5. **El inventario previo al código se pagó solo.** Los defectos se relevaron leyendo el código antes de escribir un test. Eso permitió diseñar casos que apuntan a fallos concretos, en lugar de improvisar aserciones.
6. **Documentar lo que no se prueba vale tanto como probarlo.** Se dejaron por escrito los defectos no observables desde la interfaz pública, cada uno con la razón técnica de por qué no es observable. Inventar aserciones que pasan siempre habría dado una falsa sensación de cobertura.
7. **El rojo también es un resultado.** De los 36 casos en rojo contra el código provisto, 32 corresponden a defectos corregidos y 4 a los que la consigna 3 exige demostrar sin corregir. El estado final de la suite completa es intencionalmente rojo en esos cuatro casos, y verde en los otros 69.

7. Dificultades y limitaciones

Situación	Cómo se resolvió
El proyecto no compilaba	Se corrigieron los dos errores de compilación como prerrequisito y quedaron documentados
`assert` desactivables por la JVM	Se adoptó un contrato explícito con excepciones y se dejó registrado el contraste
Connector/J no soporta MariaDB oficialmente	Se asumió el riesgo y **no se materializó**: las 34 pruebas de integración corren sin inconvenientes contra MariaDB 13.0.2
`TiempoMuerto` duerme hasta 8 s al azar	La concurrencia se prueba sobre los métodos `synchronized` con hilos propios, sin pasar por esa clase
`INTERVALO_RETORNO` sin costura de inyección	El caso de `RetornoAutomatico` espera 12 s; se documenta como limitación de testabilidad
El flag `ocupado` se gestiona fuera de los estados	No hay un caso concreto que falle; se documenta como observación de diseño y no se inventa una aserción
El caso C3 es una condición de carrera	Se implementó con dos hilos compitiendo durante un intervalo acotado y se declara como prueba probabilística
Defectos no observables desde la interfaz pública	Se documentan por inspección, con su justificación
La captura roja de las consignas 1 y 2 se obtiene revirtiendo las correcciones	Se documenta el procedimiento en el apéndice para que la evidencia sea reproducible

---

Apéndice A — Capturas de ejecución

A.1 Suite en rojo contra el código provisto (consignas 1 y 2)

Corrida de las consignas 1 y 2 contra el código **sin las correcciones** del ítem 1 y del módulo de persistencia, tal como lo entregó la cátedra. Se ejecutan las tres suites en una sola corrida: **57 casos, 32 en rojo** (30 fallas y 2 errores), con `BUILD FAILURE`.

```
$ mvn -f TPE2026/pom.xml test -Dtest=AsociadoDTOTest,BaseDeDatosTest,DataAccessObjectTest
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running persistencia.AsociadoDTOTest
[ERROR] Tests run: 23, Failures: 15, Errors: 0, Skipped: 0, Time elapsed: 0.056 s <<< FAILURE! -- in persistencia.AsociadoDTOTest
[INFO] Running persistencia.BaseDeDatosTest
[ERROR] Tests run: 18, Failures: 7, Errors: 0, Skipped: 0, Time elapsed: 0.584 s <<< FAILURE! -- in persistencia.BaseDeDatosTest
[INFO] Running persistencia.DataAccessObjectTest
[ERROR] Tests run: 16, Failures: 8, Errors: 2, Skipped: 0, Time elapsed: 0.339 s <<< FAILURE! -- in persistencia.DataAccessObjectTest
[ERROR] Errors:
[ERROR]   DataAccessObjectTest.testActualizarModificaLaFilaDelDniIndicado_I17:79 » SQLIntegrityConstraintViolation Duplicate entry '11223344
' for key 'DNI'
[ERROR]   DataAccessObjectTest.testCargarDevuelveElAsociadoIndicado_I16:71 » SQL Before start of result set
[INFO]
[ERROR] Tests run: 57, Failures: 30, Errors: 2, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[ERROR] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  33.742 s
[INFO] Finished at: 2026-11-10T20:44:27-03:00
[INFO] ------------------------------------------------------------------------
```

A.2 Suite en verde tras las correcciones (consignas 1 y 2)

La misma corrida de la §A.1, con el código de producción ya corregido: **57 casos, 0 fallas**, con `BUILD SUCCESS`. Se ejecuta con el filtro que aísla las consignas 1 y 2, porque la consigna 3 queda en rojo por diseño.

```
$ mvn -f TPE2026/pom.xml test -Dtest=AsociadoDTOTest,BaseDeDatosTest,DataAccessObjectTest
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running persistencia.AsociadoDTOTest
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in persistencia.AsociadoDTOTest
[INFO] Running persistencia.BaseDeDatosTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.801 s -- in persistencia.BaseDeDatosTest
[INFO] Running persistencia.DataAccessObjectTest
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.466 s -- in persistencia.DataAccessObjectTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 57, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  33.604 s
[INFO] Finished at: 2026-11-10T20:45:17-03:00
[INFO] ------------------------------------------------------------------------
```

A.3 Suite en rojo de la consigna 3

La corrida completa (**73 casos, 4 fallas**), dividida en cuatro partes por su extensión. La consigna 3 no se corrige, de modo que estos cuatro casos quedan en rojo **a propósito**.

Parte 1 — comando, arranque del build y el resultado de cada una de las cinco suites:

```
$ mvn -f TPE2026/pom.xml test
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< org.example:ClinicaManager_TPG >-------------------
[INFO] Building ClinicaManager_TPG 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] --- surefire:3.2.5:test (default-test) @ ClinicaManager_TPG ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running persistencia.AsociadoDTOTest
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.095 s -- in persistencia.AsociadoDTOTest
[INFO] Running persistencia.BaseDeDatosTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.246 s -- in persistencia.BaseDeDatosTest
[INFO] Running persistencia.DataAccessObjectTest
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.699 s -- in persistencia.DataAccessObjectTest
[INFO] Running modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest
[ERROR] Tests run: 7, Failures: 3, Errors: 0, Skipped: 0, Time elapsed: 16.52 s <<< FAILURE! -- in modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest
[INFO] Running modelo.modeloDominio.ambulancia.AmbulanciaStateMockTest
[ERROR] Tests run: 9, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.001 s <<< FAILURE! -- in modelo.modeloDominio.ambulancia.AmbulanciaStateMockTest
```

Parte 2 — los tres fallos de concurrencia (C3, C4 y C5) con sus trazas:

```
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testAgregarAsociadosMientrasSeVerificaLaSimulacion_C3 -- Time elapsed: 2.
633 s <<< FAILURE!
java.lang.AssertionError: Recorrer la lista de asociados mientras se la modifica no deberia fallar: java.util.ConcurrentModificationExceptio
n
at org.junit.Assert.fail(Assert.java:89)
at modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testAgregarAsociadosMientrasSeVerificaLaSimulacion_C3(AmbulanciaConcurrenciaT
est.java:126)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
at java.lang.reflect.Method.invoke(Method.java:498)
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testRetornoAutomaticoInvocaElRetornoConLaSimulacionActiva_C4 -- Time elap
sed: 12.08 s <<< FAILURE!
Wanted but not invoked:
ambulancia.retornoAutomatico(
<any modelo.modeloDominio.ambulancia.RetornoAutomatico>
);
-> at modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testRetornoAutomaticoInvocaElRetornoConLaSimulacionActiva_C4(AmbulanciaConc
urrenciaTest.java:140)

However, there was exactly 1 interaction with this mock:
ambulancia.isSimulacionActiva();
-> at modelo.modeloDominio.ambulancia.RetornoAutomatico.run(RetornoAutomatico.java:25)


at modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testRetornoAutomaticoInvocaElRetornoConLaSimulacionActiva_C4(AmbulanciaConcur
renciaTest.java:140)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testElRetornoDeTrasladarPacienteVuelveSinPaciente_C5 -- Time elapsed: 1.0
01 s <<< FAILURE!
java.lang.AssertionError: Debe volver a la clinica sin paciente, pero quedo en RegresandoDelTallerState
at org.junit.Assert.fail(Assert.java:89)
at org.junit.Assert.assertTrue(Assert.java:42)
at modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testElRetornoDeTrasladarPacienteVuelveSinPaciente_C5(AmbulanciaConcurrenciaTe
st.java:150)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
```

Parte 3 — el fallo de mocks (M7) con su traza:

```
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaStateMockTest.testRetornoAutomaticoNoCambiaElEstadoSiLaAmbulanciaEstaOcupada_M7 -- Time el
apsed: 0 s <<< FAILURE!
java.lang.AssertionError: El retorno automatico no debe actuar sobre una ambulancia ocupada expected same:<modelo.modeloDominio.ambulancia.A
tendiendoEnDomicilioState@17cdf2d0> was not:<modelo.modeloDominio.ambulancia.RegresandoClinicaSinPacienteState@1755e85b>
at org.junit.Assert.fail(Assert.java:89)
at org.junit.Assert.failNotSame(Assert.java:829)
at org.junit.Assert.assertSame(Assert.java:772)
at modelo.modeloDominio.ambulancia.AmbulanciaStateMockTest.testRetornoAutomaticoNoCambiaElEstadoSiLaAmbulanciaEstaOcupada_M7(AmbulanciaStat
eMockTest.java:102)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
```

Parte 4 — el bloque de resultados con los cuatro casos en rojo, el total de la suite y el `BUILD FAILURE`:

```
[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   AmbulanciaConcurrenciaTest.testAgregarAsociadosMientrasSeVerificaLaSimulacion_C3:126 Recorrer la lista de asociados mientras se la
modifica no deberia fallar: java.util.ConcurrentModificationException
[ERROR]   AmbulanciaConcurrenciaTest.testElRetornoDeTrasladarPacienteVuelveSinPaciente_C5:150 Debe volver a la clinica sin paciente, pero qu
edo en RegresandoDelTallerState
[ERROR]   AmbulanciaConcurrenciaTest.testRetornoAutomaticoInvocaElRetornoConLaSimulacionActiva_C4:140
Wanted but not invoked:
ambulancia.retornoAutomatico(
<any modelo.modeloDominio.ambulancia.RetornoAutomatico>
);
-> at modelo.modeloDominio.ambulancia.AmbulanciaConcurrenciaTest.testRetornoAutomaticoInvocaElRetornoConLaSimulacionActiva_C4(AmbulanciaConc
urrenciaTest.java:140)

However, there was exactly 1 interaction with this mock:
ambulancia.isSimulacionActiva();
-> at modelo.modeloDominio.ambulancia.RetornoAutomatico.run(RetornoAutomatico.java:25)


[ERROR]   AmbulanciaStateMockTest.testRetornoAutomaticoNoCambiaElEstadoSiLaAmbulanciaEstaOcupada_M7:102 El retorno automatico no debe actuar
sobre una ambulancia ocupada expected same:<modelo.modeloDominio.ambulancia.AtendiendoEnDomicilioState@17cdf2d0> was not:<modelo.modeloDomi
nio.ambulancia.RegresandoClinicaSinPacienteState@1755e85b>
[INFO]
[ERROR] Tests run: 73, Failures: 4, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[ERROR] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  53.517 s
[INFO] Finished at: 2026-11-10T20:48:20-03:00
[INFO] ------------------------------------------------------------------------
```

A.4 Cómo se reproduce la captura roja de las consignas 1 y 2

Las correcciones del código de producción están aplicadas en el repositorio, de modo que para volver al estado rojo hay que **restaurar los tres archivos de producción en su versión original** y correr la suite contra ellos. La referencia de esa versión original es el commit `983dea4`, que contiene `AsociadoDTO.java`, `BaseDeDatos.java` y `DataAccessObject.java` tal como los entregó la cátedra.

```
# desde la raíz del repositorio
git restore --source=983dea4 -- \
  TPE2026/src/main/java/persistencia/AsociadoDTO.java \
  TPE2026/src/main/java/persistencia/BaseDeDatos.java \
  TPE2026/src/main/java/persistencia/DataAccessObject.java

mvn -f TPE2026/pom.xml test -Dtest=AsociadoDTOTest,BaseDeDatosTest,DataAccessObjectTest
```

La corrida debe terminar con `Tests run: 57, Failures: 30, Errors: 2` y `BUILD FAILURE`. Terminado el procedimiento, se restituyen las versiones corregidas:

```
git restore -- TPE2026/src/main/java/persistencia/
```

El procedimiento está acotado a los **tres archivos de producción**: la suite de pruebas no se toca, y con ella se conserva el patrón de verificación de excepciones. Una variante equivalente es `git checkout 983dea4 -- <rutas>` en lugar de `git restore --source=…`.

Apéndice B — Resultados de las corridas

Todas las cifras de este informe provienen de ejecuciones reales de `mvn test`. Las dos órdenes utilizadas son:

```
# consignas 1 y 2 (las tres suites de persistencia)
mvn -f TPE2026/pom.xml test -Dtest=AsociadoDTOTest,BaseDeDatosTest,DataAccessObjectTest

# suite completa, con la consigna 3 incorporada
mvn -f TPE2026/pom.xml test
```

El resumen, en el orden en que se hicieron las corridas:

Corrida	Alcance	Resultado	Salida de consola
Consignas 1 y 2 contra el código provisto	Las tres suites de persistencia	57 casos, **32 en rojo** (30 fallas y 2 errores)	`evidencia/rojo-consignas-1-y-2.txt`
Consignas 1 y 2 tras las correcciones	Las tres suites de persistencia	57 casos, **0 en rojo**	`evidencia/verde-consignas-1-y-2.txt`
Suite completa, con la consigna 3 incorporada	Las cinco suites, sin filtro	73 casos, **4 en rojo** (M7, C3, C4 y C5)	`evidencia/rojo-consigna-3.txt`

Las capturas de ejecución de las tres corridas están en el Apéndice A. El directorio de evidencia contiene las salidas de las tres corridas de la entrega, con el detalle por suite.